

MANUAL DE IMPLANTACIÓN

Seguimiento de Pendientes

Sistema de control de proyectos, requerimientos e incidencias. Toma el archivo semanal de pendientes, arma el tablero solo y deja registro de cada cambio con nombre y fecha.

VERSIÓN	FECHA	PLATAFORMA	DESPLIEGUE
1.0	28 · 08 · 2026	Django + PostgreSQL	Docker Compose
DIRIGIDO A			
Equipo de implantación			

CONTENIDO

- | | | | |
|----|--------------------------------|----|-----------------------------|
| 01 | Qué hace el sistema | 06 | Alta de usuarios del equipo |
| 02 | Sistema de usuarios | 07 | Verificación posterior |
| 03 | Rutas de acceso y credenciales | 08 | Operación y respaldos |
| 04 | Requisitos previos | 09 | Solución de problemas |
| 05 | Despliegue paso a paso | | |

Qué hace el sistema

El área de tecnología lleva el control de sus proyectos, requerimientos e incidencias en un archivo de Excel que se actualiza cada semana. Ese archivo cumple para registrar, pero no permite ver el panorama, no dice quién cambió qué, y solo lo puede editar una persona a la vez.

Seguimiento de Pendientes toma ese mismo archivo y lo convierte en un sistema web multiusuario. Nadie cambia su forma de trabajar: el archivo semanal se sigue armando igual y se carga al sistema con un botón.

Las tres funciones

FUNCIÓN	QUÉ RESUELVE
Carga y edición	Varias estaciones cargan y editan al mismo tiempo. El archivo semanal se importa con un botón; también se puede dar de alta o editar un registro a mano desde un formulario.
Tablero automático	Los gráficos se arman solos con los datos cargados: distribución por tipo, por estatus y carga de trabajo por responsable, más el porcentaje de avance general.
Ficha de detalle	Cada registro de la tabla abre una ficha con los ocho campos completos y el historial de todos sus cambios.

Los ocho campos de cada registro

CAMPO	CONTENIDO
Nombre	Título del proyecto, requerimiento o incidencia
Número de requerimiento	Identificador único. Lo genera el sistema: PRY-001 , REQ-001 , INC-001
Fecha solicitada	Fecha de ingreso de la solicitud
Funcional solicitante	Área o persona que pide el requerimiento
Descripción	Detalle técnico o funcional
Porcentaje de avance	Valor de 0 a 100, con barra de progreso
Asignado a	Quién lo atiende. Admite varios responsables separados por guion
Observación	Comentarios adicionales o bloqueos

Cómo se procesa el archivo semanal

Los registros se identifican por **nombre y tipo**. Al cargar un archivo el sistema decide qué hacer con cada fila:

SI EL REGISTRO...	EL SISTEMA...
no existe	lo crea con código automático
existe y cambió	lo actualiza y anota cada campo en el historial
existe y está igual	no hace nada: el historial queda limpio
trae un campo vacío	conserva lo que ya estaba cargado a mano

SIN PÉRDIDA DE DATOS

Ninguna carga borra registros. Volver a subir el mismo archivo es seguro: el sistema detecta que nada cambió y no genera movimientos.

Particularidades del archivo que el sistema resuelve

El importador contempla tres detalles del archivo original que romperían una lectura directa:

- Registros partidos en dos filas.** Cuando un nombre largo se corta en dos renglones del Excel, el sistema los une en un solo registro.
- El avance viene como fracción.** Excel guarda 95% como 0,95 ; el símbolo de porcentaje es solo formato de pantalla. El sistema lo convierte a un entero de 0 a 100.
- Estatus con texto adherido.** Valores como «Calidad » con espacio sobrante o «Desarrollo 27/8/2026» con la fecha pegada se normalizan. Sin esto el tablero mostraría seis estatus donde en realidad hay cuatro.

Se aceptan archivos .xlsx , .xls y .docx . El formato .docx corresponde a los informes anteriores, con tres tablas separadas.

SECCIÓN 02

Sistema de usuarios

El sistema usa la autenticación nativa de Django: usuarios, sesiones y contraseñas cifradas. Para entrar hay que iniciar sesión; no existe acceso anónimo.

Dos niveles de acceso

NIVEL	PUEDA	SE DEFINE
Usuario	Entrar al tablero, cargar archivos, crear y editar registros, consultar el historial	Marcando solo Activo
Administrador	Todo lo anterior, más crear usuarios y consultar la auditoría completa	Marcando Activo , Es staff y Es superusuario

UNA CUENTA POR PERSONA

El sistema registra **qué usuario cargó cada archivo y qué usuario modificó cada campo**. Si el equipo comparte una sola cuenta, esa trazabilidad se pierde y la auditoría deja de servir. Crear una cuenta por persona es parte de la implantación, no un paso opcional.

Qué queda registrado

Cada acción deja tres rastros, todos consultables desde el modo administrativo:

REGISTRO	CONTENIDO
Cargas de archivo Import batches	Nombre del archivo, huella digital, fecha, usuario y cuántos registros creó, actualizó u omitió
Historial de cambios Ticket histories	Campo modificado, valor anterior, valor nuevo, usuario y fecha
Registros Tickets	Estado actual de cada proyecto, requerimiento o incidencia

Las dos primeras tablas son de **solo lectura** en el modo administrativo: no se pueden editar ni borrar desde la interfaz, porque constituyen la evidencia de auditoría.

Baja de un usuario

Cuando alguien deja el equipo, **no se borra su cuenta**: borrarla dejaría su historial sin autor. En el modo administrativo se abre el usuario y se **desmarca Activo**. Pierde el acceso de inmediato y todo su rastro queda intacto.

SECCIÓN 03

Rutas de acceso y credenciales

Reemplazar `SERVIDOR` por la dirección IP o el nombre del equipo donde queda instalado el sistema.

SISTEMA · TABLERO

`http://SERVIDOR:8000/`

Usuario `admin`

Clave `admin2026`

Tablero, tabla de registros, carga de archivos, alta y edición.

MODO ADMINISTRATIVO

`http://SERVIDOR:8000/admin/`

Usuario `admin`

Clave `admin2026`

Alta de usuarios y consulta de la auditoría completa.

Es la misma credencial para ambos accesos. `admin` es un único usuario con permiso de administrador: entra al tablero y al modo administrativo con la misma clave. Desde el tablero se llega al modo administrativo con el ícono de escudo, arriba a la derecha, visible solo para cuentas con ese permiso.

CREDENCIAL DE INSTALACIÓN

`admin` / `admin2026` es la credencial que trae el sistema de fábrica, pensada solo para el primer arranque. Se define en el archivo `.env` antes de instalar y debe cambiarse al terminar (paso 8 del despliegue). Este manual circula por correo y carpetas compartidas: mientras la clave de fábrica siga activa, cualquiera que lea este documento entra al sistema.

Rutas internas del sistema

RUTA	FUNCIÓN
/	Tablero principal con gráficos y tabla
/login/	Inicio de sesión
/logout/	Cierre de sesión
/importar/	Recepción del archivo semanal
/ticket/<n>/	Ficha de detalle de un registro
/admin/	Modo administrativo

El puerto 8000 es el valor por omisión y se cambia con APP_PORT en el archivo .env .

SECCIÓN 04

Requisitos previos

El sistema corre íntegramente en contenedores. **No hace falta instalar Python, PostgreSQL ni Node** en el servidor: Docker se encarga de todo.

Del servidor

REQUISITO	MÍNIMO	CÓMO VERIFICARLO
Docker Engine	24.0	<code>docker --version</code>
Docker Compose	v2.20	<code>docker compose version</code>
Memoria libre	2 GB	<code>free -h</code>
Disco libre	3 GB	<code>df -h</code>
Puerto de la aplicación	8000	<code>lsof -i :8000</code>
Puerto de la base	55432	<code>lsof -i :55432</code>
Sistema operativo	Linux, macOS o Windows con Docker Desktop	

Si alguno de los puertos está ocupado, se cambia en el paso 3 del despliegue.

De red

- Dirección IP fija o nombre de equipo en la red interna
- Puerto `8000/tcp` abierto en el cortafuegos hacia las estaciones
- Salida a internet **solo durante la instalación**, para descargar las imágenes de Docker

De las estaciones de trabajo

Únicamente un navegador moderno: Chrome, Edge, Firefox o Safari. **No se instala nada** en las estaciones.

Componentes que se instalan

COMPONENTE	VERSIÓN	FUNCIÓN
Python	3.12	Lenguaje base
Django	5.1.4	Marco de trabajo web y autenticación
PostgreSQL	16	Base de datos
Gunicorn	23.0	Servidor de aplicación
WhiteNoise	6.8	Entrega de archivos estáticos
openpyxl	3.1	Lectura de archivos Excel
python-docx	1.1	Lectura de archivos Word

Instalación de Docker, si hiciera falta

UBUNTU · DEBIAN

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker
```

RHEL · CENTOS · ROCKY

```
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo \
  https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

En Windows y macOS se instala Docker Desktop desde `docker.com` .

Despliegue paso a paso

Ocho pasos. La primera instalación toma entre cinco y diez minutos, casi todo descarga de imágenes.

1 Copiar el proyecto al servidor

```
scp -r pendientes-tracker/ usuario@servidor:/opt/
```

O bien, si el equipo lo versiona:

```
cd /opt  
git clone <url-del-repositorio> pendientes-tracker
```

2 Verificar los requisitos

```
docker --version          # 24.0 o superior  
docker compose version    # v2.20 o superior  
lsof -i :8000 -i :55432    # sin resultado = puertos libres
```

Averiguar la dirección del servidor, que hace falta en el paso siguiente:

```
hostname -I | awk '{print $1}' # Linux  
ipconfig getifaddr en0        # macOS
```

3

Configurar el entorno

```
cd /opt/pendientes-tracker
cp .env.example .env
```

Generar una clave secreta:

```
docker run --rm python:3.12-slim \
  python -c "import secrets; print(secrets.token_urlsafe(50))"
```

Editar `.env` con esa clave y los valores del entorno:

```
# Clave generada en el comando anterior
SECRET_KEY=<pegar-la-clave>
DEBUG=False

# IP o nombre del servidor. No dejar * en producción.
ALLOWED_HOSTS=192.168.1.50,pendientes.empresa.local

# Necesario para que los formularios funcionen al entrar por IP.
# Incluir todas las direcciones de acceso.
CSRF_TRUSTED_ORIGINS=http://192.168.1.50:8000

# Contraseña de la base de datos
DB_PASSWORD=<contraseña-fuerte>

# Cuenta de administrador que se crea sola al primer arranque
ADMIN_USER=admin
ADMIN_PASSWORD=<contraseña-fuerte>
ADMIN_EMAIL=sopORTE@empresa.com

# Puertos
APP_PORT=8000
DB_PORT_HOST=55432
```

Restringir los permisos del archivo, que contiene credenciales:

```
chmod 600 .env
```

SOBRE CSRF_TRUSTED_ORIGINS

Si esta línea falta, el sistema abre pero **ningún formulario funciona** al entrar por dirección IP. Es el error más común de esta instalación. Debe incluir todas las direcciones por las que se acceda, con su protocolo y puerto.

4 Levantar el sistema

```
docker compose up -d --build
```

El sistema descarga las imágenes, crea la base, aplica la estructura de tablas y genera el usuario administrador. Seguir el arranque:

```
docker compose logs -f web
```

El arranque terminó bien cuando aparecen estas cuatro líneas:

```
[entrypoint] postgres listo.  
Applying tracker.0001_initial... OK  
Usuario 'admin' creado.  
[INFO] Listening at: http://0.0.0.0:8000
```

5 Abrir el puerto en el cortafuegos

```
sudo ufw allow 8000/tcp # Ubuntu  
  
sudo firewall-cmd --permanent --add-port=8000/tcp # RHEL  
sudo firewall-cmd --reload
```

Sin este paso el sistema funciona en el servidor pero no se ve desde las estaciones.

6 Comprobar el acceso

```
docker compose ps # los dos contenedores en pie  
curl -I http://localhost:8000/login/ # esperado: 200 OK
```

Desde una estación de trabajo, abrir `http://SERVIDOR:8000` e ingresar con la credencial del paso 3.

7 Cargar el archivo semanal

En el tablero, botón **Cargar archivo**, seleccionar el `.xlsx` de pendientes y confirmar. Al terminar aparece el resumen: cuántos registros se crearon, cuántos se actualizaron y cuántos quedaron sin cambios.

Verificar que la cantidad coincida con las filas del archivo. Si algún registro quedó fuera, el modo administrativo lo explica en **Import batches**, columna *detalle*.

8

Cambiar la contraseña de fábrica

Este paso cierra la instalación. Mientras la clave de fábrica siga activa, cualquiera que tenga este manual entra al sistema.

```
docker compose exec web python manage.py changepassword admin
```

Actualizar también `ADMIN_PASSWORD` en el archivo `.env`, para que una reinstalación futura no reponga la clave anterior.

SECCIÓN 06

Alta de usuarios del equipo

Se hace desde el modo administrativo, en `http://SERVIDOR:8000/admin/`. También se llega desde el tablero con el ícono de escudo, arriba a la derecha.

Desde el modo administrativo

1. En **Autenticación y autorización** → **Usuarios**, pulsar **Añadir usuario**.
2. Completar **Nombre de usuario** y **Contraseña** (dos veces). Guardar.
3. Se abre la pantalla de edición. Completar nombre, apellido y correo.
4. Marcar los permisos según el nivel de acceso.
5. Guardar.

CASILLA	QUÉ HABILITA	A QUIÉN
Activo	Iniciar sesión en el tablero	A todos. Se desmarca para dar de baja
Es staff	Entrar al modo administrativo	Solo a quien administre usuarios
Es superusuario	Todos los permisos	Solo al responsable del sistema

PARA UN USUARIO COMÚN

Marcar **únicamente** «Activo». Con eso entra al tablero, carga archivos y edita registros. No necesita nada más, y no ve el modo administrativo.

Desde la terminal

Usuario común, sin acceso al modo administrativo:

```
docker compose exec web python manage.py shell -c "  
from django.contrib.auth.models import User  
User.objects.create_user('jperez', 'jperez@empresa.com', 'ClaveInicial2026')  
print('usuario creado')  
"
```

Administrador:

```
docker compose exec web python manage.py createsuperuser
```

Restablecer una contraseña olvidada:

```
docker compose exec web python manage.py changepassword jperez
```

SECCIÓN 07

Verificación posterior

Lista de comprobación antes de entregar el sistema a los usuarios. Los puntos marcados como **críticos** son de seguridad: sin ellos el sistema no debe salir a producción.

SEGURIDAD

- ☐ `SECRET_KEY` generada, distinta de la de ejemplo **CRÍTICO**
- ☐ `DEBUG=False` en el archivo `.env` **CRÍTICO**
- ☐ Contraseña de `admin` cambiada, ya no es la de fábrica **CRÍTICO**
- ☐ `DB_PASSWORD` cambiada por una contraseña fuerte **CRÍTICO**
- ☐ `ALLOWED_HOSTS` con la IP o el dominio real, sin `*`
- ☐ `chmod 600 .env` aplicado

FUNCIONAMIENTO

- ☐ Los dos contenedores en pie: `docker compose ps` muestra `db` en *healthy*
- ☐ La pantalla de inicio de sesión abre desde el servidor
- ☐ La pantalla de inicio de sesión abre desde una estación de trabajo
- ☐ El ingreso funciona y lleva al tablero
- ☐ El archivo semanal se carga y el resumen coincide con las filas del archivo
- ☐ Los tres gráficos muestran datos
- ☐ El botón **Ver detalle** abre la ficha con los ocho campos
- ☐ La ficha muestra el historial con el nombre de quien hizo la carga
- ☐ Crear un registro a mano funciona y aparece en la tabla
- ☐ Editar un registro genera una línea nueva en su historial
- ☐ Los filtros de tipo y estatus devuelven los registros correctos

USUARIOS

- ☐ Una cuenta creada por cada persona del equipo
- ☐ Cada persona ingresó al menos una vez con su cuenta
- ☐ Un usuario común **no** ve el modo administrativo
- ☐ La auditoría muestra el nombre correcto en **Import batches**

CONTINUIDAD

- ☐ Respaldo automático configurado **CRÍTICO**
- ☐ Restauración probada en un entorno de prueba **CRÍTICO**
- ☐ El sistema vuelve solo tras reiniciar el servidor
- ☐ Dirección de acceso comunicada al equipo
- ☐ Responsable del sistema designado

SOBRE EL RESPALDO

Un respaldo que nunca se restauró no es un respaldo: es un archivo. Probar la restauración en un entorno de prueba antes de dar la implantación por cerrada.

Comprobación de la auditoría

Para confirmar que el registro de cargas quedó operativo, ejecutar en el servidor:

```
docker compose exec db psql -U pendientes -d pendientes -c "
SELECT b.creado_en, b.archivo, u.username AS usuario, b.creados
FROM tracker_importbatch b
LEFT JOIN auth_user u ON u.id = b.usuario_id
ORDER BY b.creado_en DESC;"
```

El resultado debe mostrar la carga del paso 7 con el nombre de usuario en la columna correspondiente. Si aparece vacía, la carga se hizo sin sesión iniciada y hay que revisar la configuración.

SECCIÓN 08

Operación y respaldos

Comandos de uso diario

COMANDO	ACCIÓN
<code>docker compose ps</code>	Estado de los contenedores
<code>docker compose logs -f web</code>	Registro de actividad en vivo
<code>docker compose restart web</code>	Reiniciar tras cambiar <code>.env</code>
<code>docker compose stop</code>	Detener. Los datos quedan
<code>docker compose up -d</code>	Volver a levantar

Actualizar a una versión nueva

```
cd /opt/pendientes-tracker
docker compose down
git pull                # o copiar los archivos nuevos
docker compose up -d --build # la estructura se actualiza sola
docker compose logs -f web
```

Respaldo

Manual:

```
docker compose exec -T db pg_dump -U pendientes pendientes \
> respaldo_$(date +%F).sql
```

Restauración:

```
cat respaldo_2026-08-28.sql | \
docker compose exec -T db psql -U pendientes -d pendientes
```

Automático, a las tres de la mañana, conservando treinta días:

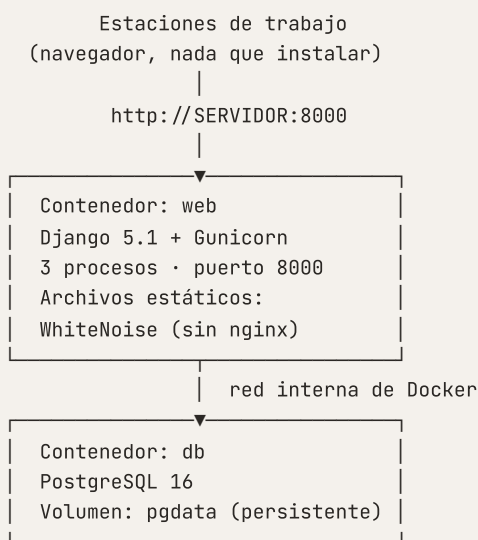
```
sudo crontab -e
```

```
0 3 * * * cd /opt/pendientes-tracker && \
  docker compose exec -T db pg_dump -U pendientes pendientes \
  > /var/backups/pendientes_$(date +%F).sql 2>/dev/null; \
  find /var/backups -name 'pendientes_*.sql' -mtime +30 -delete
```

Dónde viven los datos

En el volumen de Docker `pendientes-tracker_pgdata`, fuera de los contenedores. Reconstruir la imagen o actualizar el código **no los toca**. Solo los elimina el comando `docker compose down -v`, que debe usarse con cuidado.

Arquitectura



Detección de cambios entre estaciones

No hay servicios adicionales. Cada pantalla consulta cada quince segundos la fecha del registro más reciente. Si es posterior a la que tenía, muestra el aviso «Otra estación cargó datos» con un botón para actualizar.

Viaja un solo dato por consulta, no la tabla completa: el costo sobre la base es despreciable incluso con veinte estaciones conectadas.

SECCIÓN 09

Solución de problemas

SÍNTOMA	CAUSA	SOLUCIÓN
<code>port is already allocated</code>	El puerto 8000 está ocupado	Cambiar <code>APP_PORT</code> en <code>.env</code> y volver a levantar
<code>DisallowedHost</code>	Falta la dirección en <code>ALLOWED_HOSTS</code>	Agregarla en <code>.env</code> y <code>docker compose restart web</code>
<code>CSRF verification failed</code>	Falta el origen en <code>CSRF_TRUSTED_ORIGINS</code>	Agregar <code>http://IP:8000</code> en <code>.env</code> y reiniciar
El contenedor <code>web</code> reinicia en bucle	Falta <code>SECRET_KEY</code> o la base no arranca	<code>docker compose logs web</code> muestra el error real
<code>password authentication failed</code>	Se cambió <code>DB_PASSWORD</code> con la base ya creada	Ver la nota siguiente
Los estilos no cargan	Faltan los archivos estáticos	<code>docker compose exec web python manage.py collectstatic --noinput</code>
No entra desde otra máquina	Cortafuegos	Abrir el puerto <code>8000/tcp</code>
El archivo no importa nada	Formato distinto al esperado	Ver Import batches en el modo administrativo, columna <i>detalle</i>

SOBRE DB_PASSWORD

PostgreSQL fija la contraseña al crear el volumen de datos. Cambiarla en `.env` después no la cambia en la base. Para cambiarla de verdad:

```
docker compose exec db psql -U pendientes -d pendientes \  
-c "ALTER USER pendientes WITH PASSWORD 'nueva-clave';"  
  
# luego actualizar DB_PASSWORD en .env  
docker compose restart web
```

REINICIO TOTAL

El comando `docker compose down -v` borra todos los datos, incluido el historial de auditoría. Usarlo únicamente en entornos de prueba o tras confirmar que existe un respaldo restaurable.

